

Run selected (1000) 11:32 PM (<1s) workspace default

```
2 CREATE SCHEMA IF NOT EXISTS EASYSQL;
3
4
5 --THE PRODUCTS TABLE
6 CREATE TABLE IF NOT EXISTS products (product_id INT, product_name STRING, price INT);
7 INSERT INTO products VALUES (1, 'Laptop', 1200), (2, 'Phone', 800), (3, 'Keyboard', 45), (4, 'Monitor', 300), (5,
  'Mouse', 25);
8
9 SELECT * FROM products;
10
11 --Question1: Classify each product by price into the three tiers below.
12 SELECT product_name, price,
13 CASE
14   WHEN price > 1000 THEN 'Expensive'
15   WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
16   WHEN price < 100 THEN 'Budget'
17 END AS price_category
18 FROM products;
```

Add parameter

product_name	price	price_category
Laptop	1200	Expensive
Phone	800	Mid-range
Keyboard	45	Budget
Monitor	300	Mid-range
Mouse	25	Budget

5 rows 0.433s runtime See performance (1) Checked for improvements Refreshed 11m ago

Run selected (1000) Just now (1s) workspace default

```
21
22 --THE ORDERS TABLE
23 CREATE TABLE IF NOT EXISTS orders (order_id INT, customer_name STRING, amount DECIMAL);
24 INSERT INTO orders VALUES (1, 'Alice', 150.00), (2, 'Bob', 560.00), (3, 'Charlie', 999.99), (4, 'Diana', 45.50),
  (5, 'Ethan', 1200.00);
25
26 SELECT * FROM orders;
27
28 --Question2: Label each order by its value.
29 SELECT customer_name, amount,
30 CASE
31   WHEN amount >= 1000 THEN 'High value'
32   WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium value'
33   WHEN amount < 500 THEN 'Low value'
34 END AS order_value_category
35 FROM orders;
```

Add parameter

customer_name	amount	order_value_category
Alice	150	Low value
Bob	560	Medium value
Charlie	1000	High value
Diana	46	Low value
Ethan	1200	High value

5 rows 1.141s runtime See performance (1) Checked for improvements Refreshed Now

Run all (1000) 2 minutes ago (1s) workspace default

```
36
37 --THE EMPLOYEES TABLE
38 CREATE TABLE IF NOT EXISTS employees (emp_id INT, emp_name STRING, department STRING, salary INT);
39 INSERT INTO employees VALUES (1, 'John', 'IT', 85000), (2, 'Sara', 'HR', 60000), (3, 'Mark', 'IT', 75000), (4,
  'Lucy', 'Finance', 95000), (5, 'Tom', 'HR', 55000);
40
41 SELECT * FROM employees;
42
43 --Question3: Categorize each employee's position level using both department & salary.
44 SELECT emp_name, department, salary,
45 CASE
46   WHEN department = 'IT' AND salary > 80000 THEN 'Senior IT'
47   WHEN department = 'HR' AND salary > 55000 THEN 'Experience HR'
48   ELSE 'Staff'
49 END AS position_level
```

Add parameter

emp_name	department	salary	position_level
John	IT	85000	Senior IT
Sara	HR	60000	Experience HR
Mark	IT	75000	Staff
Lucy	Finance	95000	Staff
Tom	HR	55000	Staff

5 rows 0.525s runtime See performance (1) Checked for improvements Refreshed 2m ago

Run selected (1000) Just now (1s) workspace: default

```
53 --THE STUDENTS TABLE
54 CREATE TABLE IF NOT EXISTS students (student_id INT, student_name STRING, score INT);
55 INSERT INTO students VALUES (1, 'Anna', 92), (2, 'Ben', 76), (3, 'Cara', 59), (4, 'David', 83), (5, 'Ella', 68);
56
57 SELECT * FROM students;
58
59 --Question4: Assign each student a letter grade based on their score.
60 SELECT student_name, score,
61 CASE
62     WHEN score >= 90 THEN 'A'
63     WHEN score BETWEEN 80 AND 89 THEN 'B'
64     WHEN score BETWEEN 70 AND 79 THEN 'C'
65     WHEN score BETWEEN 60 AND 69 THEN 'D'
66     WHEN score < 60 THEN 'F'
67 END AS grade
68 FROM students;
```

Add parameter

	student_name	score	grade
1	Anna	92	A
2	Ben	76	C
3	Cara	59	F
4	David	83	B
5	Ella	68	D

5 rows 1.108s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (1s) workspace: default

```
70
71 --THE DELIVERIES TABLE
72 CREATE TABLE IF NOT EXISTS deliveries (delivery_id INT, delivery_time_minutes INT);
73 INSERT INTO deliveries VALUES (1, 45), (2, 80), (3, 30), (4, 65), (5, 100);
74
75 SELECT * FROM deliveries;
76
77 --Question5: Label delivery performance based on the time taken
78 SELECT delivery_id, delivery_time_minutes,
79 CASE
80     WHEN delivery_time_minutes <= 30 THEN 'Fast'
81     WHEN delivery_time_minutes BETWEEN 31 AND 60 THEN 'On Time'
82     WHEN delivery_time_minutes > 60 THEN 'Late'
83 END AS performance
84 FROM deliveries;
85
```

Add parameter

	delivery_id	delivery_time_minutes	performance
1	1	45	On Time
2	2	80	Late
3	3	30	Fast
4	4	65	Late
5	5	100	Late

5 rows 0.573s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (<1s) workspace: default

```
85
86 --THE TICKETS TABLE
87 CREATE TABLE IF NOT EXISTS tickets (ticket_id INT, issue_type STRING, priority INT);
88 INSERT INTO tickets VALUES (1, 'Login issue', 1), (2, 'Server down', 3), (3, 'Slow system', 2), (4, 'Email error', 2), (5, 'Password reset', 1);
89
90 SELECT * FROM tickets;
91
92 --Question6: Convert priority to a label.
93 SELECT issue_type, priority,
94 CASE
95     WHEN priority = 3 THEN 'High'
96     WHEN priority = 2 THEN 'Medium'
97     WHEN priority = 1 THEN 'Low'
98 END AS priority_label
99 FROM tickets;
```

Add parameter

	issue_type	priority	priority_label
1	Login issue	1	Low
2	Server down	3	High
3	Slow system	2	Medium
4	Email error	2	Medium
5	Password reset	1	Low

5 rows 0.391s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (<1s) workspace default

```
100
101 --THE ATTENDANCE TABLE
102 CREATE TABLE IF NOT EXISTS attendance (student_id INT, days_present INT, total_days INT);
103 INSERT INTO attendance VALUES (1, 45, 50), (2, 30, 50), (3, 48, 50), (4, 25, 50), (5, 50, 50);
104
105 SELECT * FROM attendance;
106
107 --Question7: Calculate attendance percentage and classify the result
108 SELECT student_id, (days_present/total_days)*100 AS attendance_percentage,
109 CASE
110     WHEN (days_present/total_days)*100 >= 90 THEN 'Excellent'
111     WHEN (days_present/total_days)*100 BETWEEN 75 AND 89 THEN 'Good'
112     WHEN (days_present/total_days)*100 < 75 THEN 'Needs Improvement'
113 END AS attendance_status
114 FROM attendance;
115
```

Add parameter

Table	+
1. student_id	1.2 attendance_percentage
2.	3. attendance_status

5 rows 0.447s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (<1s) workspace default

```
115
116 --THE PRODUCTS INVENTORY TABLE
117 CREATE TABLE IF NOT EXISTS products_inventory (product_id INT, stock_quantity INT);
118 INSERT INTO products_inventory VALUES (1, 5), (2, 0), (3, 25), (4, 10), (5, 3);
119
120 SELECT* FROM products_inventory;
121
122 --Question8: Label the stock status of each product
123 SELECT product_id, stock_quantity,
124 CASE
125     WHEN stock_quantity = 0 THEN 'Out of Stock'
126     WHEN stock_quantity BETWEEN 1 AND 5 THEN 'Low Stock'
127     WHEN stock_quantity > 5 THEN 'In Stock'
128 END AS stock_status
129 FROM products_inventory;
130
```

Add parameter

Table	+
1. product_id	2. stock_quantity
3.	4. stock_status

5 rows 0.425s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (<1s) workspace default

```
130
131 --THE CLASSES TABLE
132 CREATE TABLE IF NOT EXISTS classes (class_id INT, subject STRING, enrolled_students INT);
133 INSERT INTO classes VALUES (1, 'Math', 30), (2, 'English', 25), (3, 'Science', 15), (4, 'Art', 5), (5, 'History', 20);
134
135 SELECT* FROM classes;
136
137 --Question9: Classify each class by the number of enrolled students
138 SELECT subject, enrolled_students,
139 CASE
140     WHEN enrolled_students >= 25 THEN 'Large'
141     WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
142     WHEN enrolled_students < 10 THEN 'Small'
143 END AS class_size_category
144 FROM classes;
```

Add parameter

Table	+
1. subject	2. enrolled_students
3.	4. class_size_category

5 rows 0.361s runtime See performance (1) Checked for improvements Refreshed Now

Run selected (1000) Just now (<1s) workspace. default

146 --THE PAYMENTS TABLE

147 CREATE TABLE IF NOT EXISTS payments (payments_id INT, amount DECIMAL, payment_method STRING);

148 INSERT INTO payments VALUES (1, 50.00, 'Card'), (2, 200.00, 'Cash'), (3, 150.00, 'Card'), (4, 75.00, 'PayPal'), (5, 300.00, 'Cash');

149

150 SELECT* FROM payments;

151

152 --Question10: Apply a discount flag based on the payment method and amount.

153 SELECT payments_id, payment_method, amount,

154 CASE

155 WHEN payment_method = 'Cash' AND amount >= 200 THEN 'Eligible for Discount'

156 ELSE 'Not Eligible'

157 END AS discount_eligibility

158 FROM payments;

159

160

Add parameter

Table +

payments_id

payment_method

amount

discount_eligibility

	payments_id	payment_method	amount	discount_eligibility
1	1	Card	50	Not Eligible
2	2	Cash	200	Eligible for Discount
3	3	Card	150	Not Eligible
4	4	PayPal	75	Not Eligible
5	5	Cash	300	Eligible for Discount

5 rows

0.103s runtime

See performance (1)

Checked for improvements

Refreshed Now